

Indian Institute of Technology Madras

Electronic Systems

Signal Processing Project Report

Smartphone-Based Vibration Analysis and Machinery Health Monitoring Using FFT and RMS Techniques

Author: Divyansh Verma

Abstract

This project presents a smartphone-based vibration analysis and machinery health monitoring system using Digital Signal Processing (DSP) techniques. Industrial vibration monitoring systems are often expensive and require specialized hardware. The aim of this project is to develop a low-cost vibration analysis system using a smartphone accelerometer and Python-based DSP processing.

Real-time vibration data was collected using the accelerometer sensor of a smartphone through the phyphox application. The recorded sensor data was exported as a CSV file and processed in Python using libraries such as NumPy, Pandas, and Matplotlib. The key DSP techniques used in this project are Fast Fourier Transform (FFT) and Root Mean Square (RMS) analysis. FFT was used to convert the vibration signal from the time domain to the frequency domain and identify dominant frequency components. RMS analysis was used to estimate vibration intensity and machine condition.

The system successfully generated frequency spectrum plots from real accelerometer data and demonstrated practical vibration analysis using simple and low-cost tools. The project highlights the usefulness of DSP techniques in machinery health monitoring and vibration analysis applications.

Chapter 1

Introduction

Vibration analysis is an important application of Digital Signal Processing (DSP) used in machinery health monitoring, predictive maintenance, and fault detection systems. Excessive vibration in rotating machines and mechanical systems may indicate faults such as imbalance, looseness, misalignment, or bearing wear. Early detection of such problems helps reduce maintenance cost and prevents unexpected machine failure.

Traditional vibration monitoring systems generally use dedicated sensors and expensive industrial equipment for signal acquisition and analysis. However, modern smartphones contain built-in accelerometer sensors capable of measuring motion and vibration signals. This creates an opportunity to develop low-cost vibration analysis systems using

smartphones and software-based DSP techniques.

In this project, a smartphone accelerometer was used to collect real-time vibration data. The recorded signal was processed in Python using DSP techniques such as Fast Fourier Transform (FFT) and Root Mean Square (RMS) analysis. FFT was used to analyze the frequency components of the vibration signal, while RMS analysis was used to estimate vibration intensity.

The project demonstrates a simple and cost-effective approach for vibration analysis and machinery health monitoring using commonly available devices and software tools. It also highlights the practical application of DSP concepts in real-world engineering problems.

1.1 Motivation

The motivation behind this project is to develop a simple and low-cost vibration monitoring system using easily available smartphone sensors and DSP techniques. The project also aims to demonstrate practical applications of FFT and RMS analysis in real-world signal processing.

1.2 Scope of the Project

This project focuses on vibration signal acquisition and analysis using a smartphone accelerometer and Digital Signal Processing (DSP) techniques. The main scope of the project includes collecting real-time vibration data, processing the signal using Python, performing FFT-based frequency analysis, calculating RMS vibration intensity, and generating vibration spectrum plots.

The project demonstrates a low-cost and software-based approach for machinery vibration monitoring using commonly available devices and tools. Real accelerometer data was collected using the phyphox smartphone application and processed using NumPy, Pandas, and Matplotlib libraries.

The project does not include advanced industrial fault diagnosis, machine learning-based prediction, real-time cloud monitoring, or dedicated hardware sensor systems. Development of a complete Android or iOS application is also outside the scope of this implementation. The focus is mainly on understanding and implementing practical DSP techniques for vibration analysis.

Chapter 2

Problem Statement

Industrial vibration monitoring systems are widely used for machinery health analysis and fault detection. However, most existing systems require dedicated sensors, specialized hardware, and costly vibration analysis equipment. Such systems may not be easily accessible for low-cost educational and experimental applications.

This project aims to develop a smartphone-based vibration analysis and machinery health monitoring system using Digital Signal Processing (DSP) techniques. Real-time vibration data is collected using the accelerometer sensor of a smartphone and processed in Python. FFT analysis is used to identify the frequency components present in the vibration signal, while RMS analysis is used to estimate vibration intensity and machine condition.

The project focuses on low-cost vibration analysis using software-based DSP processing. The system is designed mainly for educational demonstration and basic vibration monitoring applications. The implementation assumes that the smartphone accelerometer data is sufficient for basic vibration analysis and does not aim to replace high-precision industrial monitoring systems.

Chapter 3

Goals and Objectives

3.1 Primary Goal

The primary goal of this project is to develop a smartphone-based vibration analysis and machinery health monitoring system using Digital Signal Processing (DSP) techniques such as FFT and RMS analysis.

3.2 Specific Objectives

- To collect real-time vibration data using a smartphone accelerometer

- To process accelerometer data using Python-based DSP techniques
- To perform FFT-based frequency spectrum analysis of vibration signals
- To calculate RMS values for vibration intensity estimation
- To generate graphical visualization of vibration spectra
- To demonstrate a low-cost approach for machinery vibration monitoring

Chapter 4

Methodology

This project follows a smartphone-based vibration analysis approach using Digital Signal Processing (DSP) techniques. The methodology consists of three major stages: vibration signal acquisition, signal processing, and result visualization.

In the first stage, vibration data was collected using the accelerometer sensor of a smartphone through the phyphox application. The phone was subjected to controlled hand motion to generate vibration signals. The recorded sensor data was exported in CSV format for further analysis.

In the second stage, the exported CSV data was processed in Python using NumPy and Pandas libraries. The vibration signal was analyzed to determine the sampling frequency and signal characteristics. Fast Fourier Transform (FFT) analysis was performed to convert the signal from the time domain to the frequency domain and identify the dominant frequency components present in the vibration signal.

Root Mean Square (RMS) analysis was also performed to estimate vibration intensity and signal energy. RMS values were used as an indicator of vibration strength and machine condition.

In the final stage, Matplotlib was used to generate FFT spectrum plots for visualization of the vibration characteristics. The generated frequency spectrum and RMS values were analyzed to study the behavior of the recorded vibration signal.

4.1 System Overview

The proposed system consists of three major stages: vibration signal acquisition, signal processing, and result visualization. Real-time vibration data was collected using the accelerometer sensor of a smartphone through the phyphox application. The recorded vibration signal was exported in CSV format for further processing.

The collected data was processed in Python using NumPy, Pandas, and Matplotlib libraries. FFT analysis was performed to convert the signal from the time domain to the frequency domain and identify dominant vibration frequencies. RMS analysis was used to estimate vibration intensity and overall signal energy.

Finally, the processed results were visualized using FFT spectrum plots and terminal-based vibration analysis outputs.

Smartphone Accelerometer



CSV Data Collection



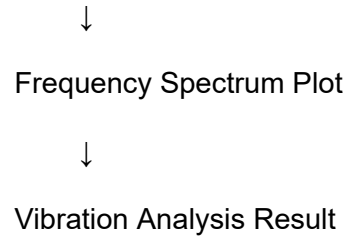
Python Signal Processing



FFT Analysis



RMS Calculation



4.2 Signal Processing Techniques Used

Digital Signal Processing (DSP) techniques were used to analyze the vibration signal obtained from the smartphone accelerometer. The primary DSP methods used in this project are Fast Fourier Transform (FFT) and Root Mean Square (RMS) analysis.

4.2.1 Filtering

No advanced filtering methods were implemented in this project. The raw accelerometer signal obtained from the smartphone sensor was directly processed for FFT and RMS analysis. Minor noise components were naturally present due to hand-generated vibration and sensor limitations.

4.2.2 Transform-Domain Analysis

FFT analysis was performed to transform the vibration signal from the time domain into the frequency domain. This helped identify the dominant frequency components present in the vibration signal. FFT is widely used in vibration monitoring systems because it provides efficient spectral analysis of signals.

For example, the Discrete Fourier Transform (DFT) is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}$$

Where $x[n]$ represents the input vibration signal and $X[k]$ represents the frequency-domain spectrum obtained after FFT processing.

4.2.3 Additional Techniques

RMS analysis was used to estimate vibration intensity from the accelerometer signal. RMS provides a measure of the average energy present in the vibration signal and is commonly used in machinery condition monitoring applications.

RMS Equation:

$$x_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{n=1}^N x[n]^2}$$

Higher RMS values indicate stronger vibration intensity in the measured signal.

4.3 Theoretical Background

Digital Signal Processing (DSP) techniques are widely used in vibration analysis and machinery health monitoring systems. FFT is an efficient algorithm used to compute the Discrete Fourier Transform (DFT) of a signal. It helps identify the frequency components present in vibration data.

RMS analysis is commonly used to estimate signal energy and vibration strength. In this project, accelerometer data collected from a smartphone was processed using FFT and RMS techniques in Python to study the characteristics of the vibration signal.

Chapter 5

Implementation Details

5.1 Tools and Technologies

The project was implemented using Python and smartphone-based sensor data acquisition tools. The following technologies and software tools were used during implementation:

- Programming Language: Python

- Libraries Used:
 - NumPy
 - Pandas
 - Matplotlib

- Sensor Platform: Smartphone Accelerometer
- Data Acquisition Application: phyphox
- Development Environment: Visual Studio Code (VS Code)
- Operating System: Windows

5.2 System Architecture

The system architecture consists of data acquisition, signal processing, and visualization modules. The smartphone accelerometer acts as the vibration sensor and records real-time vibration data. The recorded signal is exported as a CSV file and processed in Python.

The Python processing module reads the accelerometer data using Pandas and performs FFT and RMS analysis using NumPy functions. The resulting frequency spectrum is visualized using Matplotlib plots. The overall system architecture provides a simple pipeline for vibration monitoring and DSP-based signal analysis.

Smartphone Accelerometer



phyphox Application



CSV Data Export



Python Processing Script



FFT + RMS Analysis



Frequency Spectrum Plot



Vibration Analysis Output

5.3 Key Algorithms / Pseudocode

The vibration analysis process follows a sequence of signal acquisition, FFT processing, RMS calculation, and visualization.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load accelerometer CSV data
data = pd.read_csv("Raw Data.csv")

# Read time and vibration signal
time = data.iloc[:,0]
signal = data.iloc[:,4]

# Calculate sampling interval
dt = time[1] - time[0]

# Calculate sampling frequency
fs = 1 / dt

# RMS calculation
rms = np.sqrt(np.mean(signal**2))

# FFT analysis
fft = np.fft.fft(signal)

# Frequency axis
freq = np.fft.fftfreq(len(signal), d=dt)
```

```

# Positive frequencies only

positive_freq = freq[:len(freq)//2]
positive_fft = np.abs(fft[:len(fft)//2])

# Plot FFT spectrum

plt.plot(positive_freq, positive_fft)

plt.xlabel("Frequency (Hz)")
plt.ylabel("Amplitude")
plt.title("FFT Spectrum of Real Accelerometer Data")

plt.grid()

plt.show()

# Display vibration condition
if rms > 1:
    print("High Vibration")
else:
    print("Normal Vibration")

```

The above algorithm loads accelerometer vibration data from a CSV file, calculates sampling frequency, performs RMS analysis, applies FFT for frequency-domain analysis, and generates the FFT spectrum plot for vibration analysis.

5.4 User Interface

The project does not use a dedicated graphical user interface or mobile application. The implementation is based on Python scripts executed in Visual Studio Code. FFT spectrum plots were displayed using Matplotlib visualization windows, and vibration analysis results such as RMS value and sampling frequency were displayed in the terminal output window.

Screenshots of the FFT graph, CSV data, and terminal output are included in the Results

Chapter 6

Problems Faced and Solutions

6.1 Problem 1: [Noisy Accelerometer Signal]

Issue: The accelerometer signal collected from the smartphone contained irregular fluctuations and noise due to hand-generated vibration and sensor limitations. This produced noisy FFT spectra with multiple unwanted frequency components.

Solution: Multiple vibration recordings were tested to obtain more stable signals. Controlled hand movement was used during data collection to reduce random motion and generate clearer vibration patterns for FFT analysis.

6.2 Problem 2: [Incorrect FFT Spectrum]

Issue: Initially, the FFT output showed only a single sharp frequency peak because a simulated sine-wave signal was used instead of real accelerometer data. This did not represent practical vibration signals.

Solution: The implementation was modified to process real accelerometer data collected from the smartphone sensor. After using actual sensor data, the FFT spectrum showed realistic multiple frequency components and vibration characteristics.

6.3 Problem 3: [Irregular Vibration Measurements]

Issue: The vibration signals generated through hand movement were not perfectly periodic, causing variations in FFT amplitudes and frequency components.

Solution: The vibration data was analyzed directly as a real-world non-ideal signal. RMS analysis and FFT spectrum visualization were used together to study the overall vibration behavior despite signal irregularities.

6.4 Problem 4: [CSV Data Parsing Error]

Issue: The exported accelerometer CSV file had formatting and column-handling issues, which caused errors while reading the data in Python using Pandas.

Solution: The correct CSV format was verified and the required signal columns were accessed using index-based selection methods in Python. This ensured successful reading and processing of accelerometer data.

Chapter 7

Results and Discussion

7.1 Experimental Setup

The vibration data used in this project was collected using the accelerometer sensor of a smartphone through the phyphox application. Controlled hand movement was used to generate vibration signals. The recorded sensor data was exported as a CSV file and processed in Python.

The vibration signal was analyzed using FFT and RMS techniques. NumPy and Pandas libraries were used for signal processing, while Matplotlib was used for FFT spectrum visualization. The observed sampling frequency of the accelerometer signal was approximately 122 Hz.

7.2 Quantitative Results

Metric	Observed Value
Sampling Frequency	122.32 Hz
RMS Value	2.86
Signal Source	Smartphone Accelerometer
Analysis Technique	FFT + RMS
Platform	Python

The calculated RMS value indicated noticeable vibration intensity due to hand-generated motion. FFT analysis successfully identified multiple

frequency components present in the vibration signal.

7.3 Qualitative Results

The FFT spectrum generated from the real accelerometer signal showed multiple frequency peaks and noise components corresponding to practical vibration behavior. The spectrum was significantly different from the ideal sine-wave FFT obtained during initial testing, indicating successful processing of real-world vibration data.

Screenshots of the FFT spectrum plot, accelerometer CSV data, and terminal output are included as part of the project results.

Figure 7.3.1: FFT spectrum of real accelerometer vibration data

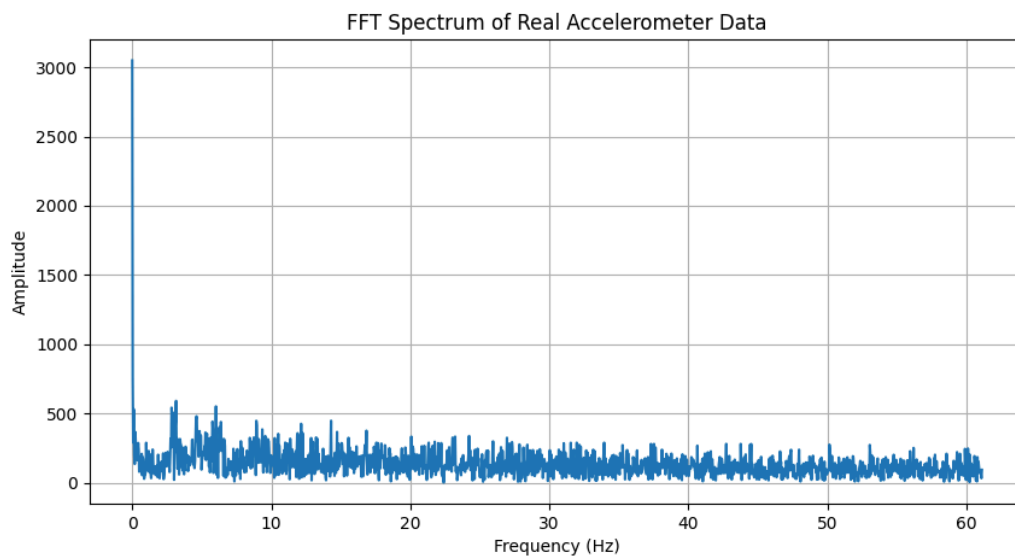


Figure 7.3.2: RMS value and sampling frequency output

```

64\python.exe c:/Users/divya/Downloads/DSP_project/vibration_analysis.py
Time (s) ... Absolute acceleration (m/s^2)
0 0.019950 ... 0.028007
1 0.028125 ... 0.023517
2 0.036302 ... 0.035793
3 0.044479 ... 0.016329
4 0.052656 ... 0.008177

[5 rows x 5 columns]
Sampling Frequency = 122.32013405308133
RMS Value = 2.8591814774152495

```

Figure 7.3.3: Real accelerometer vibration data exported in CSV format

```

1 "Time (s)","Linear Acceleration x (m/s^2)","Linear Acceleration y (m/s^2)","Linear Acceleration z (m/s^2)","Absolute acceleration (m/s^2)"
2 1.995004900E-2,4.550814629E-4,-1.010008156E-3,-2.798461914E-2,2.800653717E-2
3 2.812531800E-2,-2.232924104E-3,-2.250885963E-2,-6.435394287E-3,2.351699410E-2
4 3.630189500E-2,-2.526782453E-3,-2.964246273E-2,1.990222931E-2,3.579328090E-2
5 4.447908700E-2,-2.820648253E-3,1.531131566E-3,-1.601028442E-2,1.632879750E-2
6 5.265585600E-2,-3.114551306E-3,3.974303603E-3,-6.431579590E-3,8.176841382E-3
7 6.083258700E-2,-1.059100777E-2,4.023283720E-3,3.147125244E-3,1.175842909E-2
8 6.902977900E-2,-1.806751639E-2,-5.504526198E-3,-2.079391479E-2,2.809131271E-2
9 7.720797100E-2,-2.075633034E-2,-7.849641144E-3,7.938385010E-3,2.356820044E-2
10 8.538743300E-2,-5.494493991E-3,-5.547806621E-3,1.262950897E-2,1.484830359E-2
11 9.356558700E-2,-1.297105476E-2,6.472192705E-3,-1.131248474E-2,1.838776362E-2
12 1.017439330E-1,5.888633430E-3,8.915379643E-3,-6.523132324E-3,1.251843653E-2
13 1.099215100E-1,1.517152786E-2,1.781783998E-3,-6.522178650E-3,1.660990145E-2
14 1.180978940E-1,1.248343289E-2,-7.746048272E-3,-3.046321869E-2,3.382077843E-2
15 1.262745480E-1,1.727183908E-2,4.224956036E-3,-1.732826233E-3,1.786531180E-2
16 1.344509330E-1,2.906117588E-3,-1.233826578E-2,3.055572510E-3,1.303897408E-2
17 1.426277400E-1,-4.178043455E-3,4.371866584E-3,-8.915901184E-3,1.077323343E-2
18 1.508023560E-1,-8.966434747E-3,-2.565875649E-3,3.052711487E-3,9.813241938E-3
19 1.589927020E-1,-9.015005082E-3,-2.517081797E-3,-6.525039673E-3,1.140973970E-2
20 1.671877020E-1,-9.015005082E-3,2.271324396E-3,-8.918762207E-3,1.288307224E-2
21 1.753635860E-1,-1.140872389E-2,2.271011472E-3,3.052711487E-3,1.202645090E-2
22 1.835392790E-1,5.301713943E-3,1.668520272E-2,-1.736640930E-3,1.759318283E-2
23 1.917152400E-1,-1.831926405E-3,-7.256813347E-3,-2.089118958E-2,2.219141943E-2
24 1.998907020E-1,5.399167538E-3,-1.228973269E-4,5.446434021E-3,7.670055893E-3
25 2.080656630E-1,5.399167538E-3,-1.718774438E-4,6.580352783E-4,5.441834469E-3
26 2.162400860E-1,1.263124496E-2,2.319991589E-3,-4.131317139E-3,1.349068165E-2
27 2.244153940E-1,-1.782435924E-3,4.664950073E-3,-6.525039673E-3,8.216749950E-3
28 2.325918170E-1,-1.684010029E-3,7.058851421E-3,-6.526947021E-3,9.760343774E-3
29 2.407663940E-1,-4.274092615E-3,7.156811655E-3,2.938747406E-2,3.054687304E-2
30 2.489410480E-1,-6.471939385E-3,-2.322353423E-3,-3.286457062E-2,3.357617201E-2
31 2.571373550E-1,-6.569623947E-3,1.204258949E-2,-1.739501953E-3,1.382786727E-2
32 2.653110090E-1,-1.829218119E-3,-4.717402160E-3,1.741600037E-2,1.813606878E-2
33 2.734853170E-1,-8.865412325E-3,9.648084641E-3,-2.807617188E-2,3.098310023E-2
34 2.816600480E-1,1.483178511E-2,9.599126875E-3,2.938652039E-2,3.428837510E-2
35 2.898342780E-1,2.664420754E-3,1.692892611E-2,3.178119659E-2,3.610723105E-2
36 2.980089320E-1,-6.717018783E-3,1.453500986E-2,-5.680561066E-2,5.901916854E-2
37 3.061832010E-1,-4.567660391E-3,4.860259593E-3,3.657150269E-2,3.717472869E-2
38 3.143573550E-1,-1.977618784E-3,-1.185020059E-2,-3.286266327E-2,3.498989665E-2
39 3.225326630E-1,-1.928646117E-3,-1.639373600E-2,-2.807712555E-2,3.256991304E-2
40 3.307077780E-1,1.229015365E-2,3.053396940E-3,-1.743316650E-3,1.278320236E-2
41 3.388836240E-1,2.762325108E-3,1.267917454E-2,-8.926391602E-3,1.575031346E-2
42 3.470612780E-1,-6.716027856E-3,7.988460362E-3,-2.089977264E-2,2.336067263E-2
43 3.552581240E-1,-9.208925068E-3,-8.574247360E-3,1.022434235E-2,1.621293296E-2
44 3.634532010E-1,-1.639153063E-2,-3.834813833E-3,-6.534576416E-3,1.805792797E-2

```

7.4 Discussion

The project successfully demonstrated practical vibration analysis using smartphone accelerometer data and Digital Signal Processing (DSP) techniques. FFT analysis provided frequency-domain information about the vibration signal, while RMS analysis estimated vibration intensity and signal energy.

The generated FFT spectrum showed realistic multiple frequency components and noise characteristics due to non-ideal hand-generated vibration. This behavior matched theoretical expectations for real-world sensor signals. The implementation demonstrated that low-cost smartphone sensors can be effectively used for basic vibration monitoring and DSP experimentation.

One limitation of the current implementation is that no advanced filtering or industrial-grade fault diagnosis techniques were implemented. The vibration measurements also depended on manual hand motion, which introduced irregular signal variations.

Chapter 8

Conclusions

This project successfully implemented a smartphone-based vibration analysis and machinery health monitoring system using Digital Signal Processing (DSP) techniques. Real accelerometer data was collected using a smartphone sensor and processed in Python using FFT and RMS analysis.

The generated FFT spectrum successfully identified multiple frequency components present in the vibration signal, while RMS analysis was used to estimate vibration intensity. The project demonstrated that low-cost smartphone sensors can be effectively used for practical vibration analysis and DSP experimentation.

8.1 Summary of Contributions

The project successfully demonstrated vibration signal acquisition using a smartphone accelerometer and DSP-based signal processing using Python. FFT analysis and RMS calculation were implemented to study vibration characteristics and estimate vibration intensity. Frequency spectrum plots and real sensor data analysis were successfully generated.

8.2 Limitations

The current implementation uses smartphone accelerometer data, which may contain noise and limited sensor accuracy compared to industrial vibration monitoring systems. Advanced filtering, automatic fault diagnosis, and real-time monitoring features were not implemented in this project.

8.3 Future Work

Future improvements may include implementation of real-time vibration monitoring, advanced filtering methods, machine learning-based fault detection, wireless sensor integration, and development of a dedicated mobile or web application for vibration analysis.

Chapter 9

Individual Contribution

This is an individual project; all work was carried out by the author.

The author performed vibration data acquisition using a smartphone accelerometer, implemented FFT and RMS analysis in Python, generated frequency spectrum plots, analyzed the vibration signal, and prepared the complete project documentation and report.

Chapter 10

Annexure

10.1 Google Drive Folder Link:



The Google Drive folder contains the project source code, accelerometer dataset, FFT spectrum output, terminal results, project report, and demonstration video.

10.2 Live Deployment (if applicable)

This project was implemented as a local Python-based DSP application and was not deployed as a live web or mobile application.

References

1. A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.
2. J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed., Pearson, 2007.
3. NumPy Documentation. Available: <https://numpy.org/>
4. Pandas Documentation. Available: <https://pandas.pydata.org/>
5. Matplotlib Documentation. Available: <https://matplotlib.org/>
6. phyphox – Physical Phone Experiments Application. Available: <https://phyphox.org/>
7. Python Software Foundation. Python Documentation. Available: <https://www.python.org/>